Here with(3) was added to the double declaration, leading to this:

```
1) Project stubs the class
    Failure/Error: project = Project.find(1)
      <Project(id: integer, name: string, due_date: date, created_at: datetime,
        updated_at: datetime) (class)> received :find with unexpected arguments
        expected: (3)
             got: (1)
       Please stub a default value first if message
            might be received with other args as well.
```

This message is saying that RSpec doesn't know what to do if find is called with the argument 1, but it would know what to do with the argument 3. In other words, you did something RSpec didn't expect, and RSpec doesn't like surprises.

You can have multiple stubs of the same method that use with to expect different arguments and return different values. As implied by the last line of the preceding error message, a stub without a with argument is considered a default and is used if nothing else matches:

```
it "stubs the class" do
  allow(Project).to receive(:find).with(1).and_return(
        Project.new(:name => "Project Greenlight"))
  allow(Project).to receive(:find).with(3).and_return(
        Project.new(:name => "Project Runway"))
  allow(Project).to receive(:find).and_raise(ActiveRecord::RecordNotFound)
  project = Project.find(1)
  expect(project.name).to eq("Project Greenlight")
end
```

Using with does not constrain the eventual return value. You can use either returns or raises after a with call.

Be careful here—using with makes RSpec powerful and flexible, but in general, testing with mock objects works best if they are weak and rigid. The use of a complicated mock object suggests the existence of an overly complex dependency in your code.

Many of these matchers make more sense when you're talking about mock expectations rather than just stubs. When you're actually validating the behavior of the object being mocked, having a tighter filter on the incoming values you expect makes more sense.

RSpec allows you to use many things besides literal values as the arguments to a with call. Here are a few:

- If you expect the method to be called with no arguments, use with(no_args).